

Route Generation Module

Technical Design Specification

AR Tourism Platform — Axe Rêve Competition Project

Version 1.0 • Core route-generation logic (Layers 2–5)

Contents

1. Purpose & Scope
2. Architecture Overview
3. Layer Responsibilities
4. System Workflow & Data Flow
5. Core Algorithms & Routing Logic
6. APIs & External Services
7. Data Models & Key Interfaces
8. Feature Specifications
9. Implementation Roadmap & Development Order
10. Edge Cases & Assumptions
11. Basic Testing Strategy

1. Purpose & Scope

This document is the implementation specification for the Route Generation module — the component that turns a theme and time budget into an ordered, hybrid walking/driving route across a city's points of interest (POIs). It is written to be used directly as build context: architecture, algorithms, data models, and build order, with no unnecessary theory.

In scope

- Layers 2–5: Orchestration, Domain, Adapter, Data — the internal generation pipeline.
- Data schema, routing-provider integration, clustering and ordering algorithms, caching strategy.

Out of scope (deferred)

- Layer 1 (the HTTP API controller exposing this module to a frontend) — a minimal driver script substitutes for now (see Section 9).
- Frontend/mobile app, AR trigger implementation, gamification logic, authentication.

Assumptions

- Mobile-first app; pilot city is Algiers, with additional cities added later via config, not code changes.
- Small team, constrained budget, no international payment card currently available — routing providers are chosen accordingly (Section 6).
- Feasibility and cost are explicit jury evaluation criteria for this project; the architecture below is written with that in mind.

2. Architecture Overview

Layered modular monolith — one deployable service, internal layer boundaries only. Not microservices at this stage: the operational cost (network calls, distributed debugging, deployment orchestration) isn't justified yet. Boundaries are drawn so a future service split, if ever needed, follows existing seams instead of requiring a rewrite.

Key design decisions

- Routing engine is only ever called through a single Adapter interface (`getRoute` / `getMatrix` / `getIsochrone`). Domain logic never imports a provider SDK. This is what makes swapping providers, or self-hosting later, a one-file change.
- Hybrid transport model: driving between POI clusters, walking within a cluster — matches real tourist behavior in spread-out Algerian cities rather than assuming a single walking loop.
- Cache-first on every Adapter call: checked before any external request, both to hit the latency target and to conserve limited free-tier provider quota.
- City-level configuration (cluster radius, active provider, rollout status) drives behavior without code changes — the mechanism behind the phased city rollout.

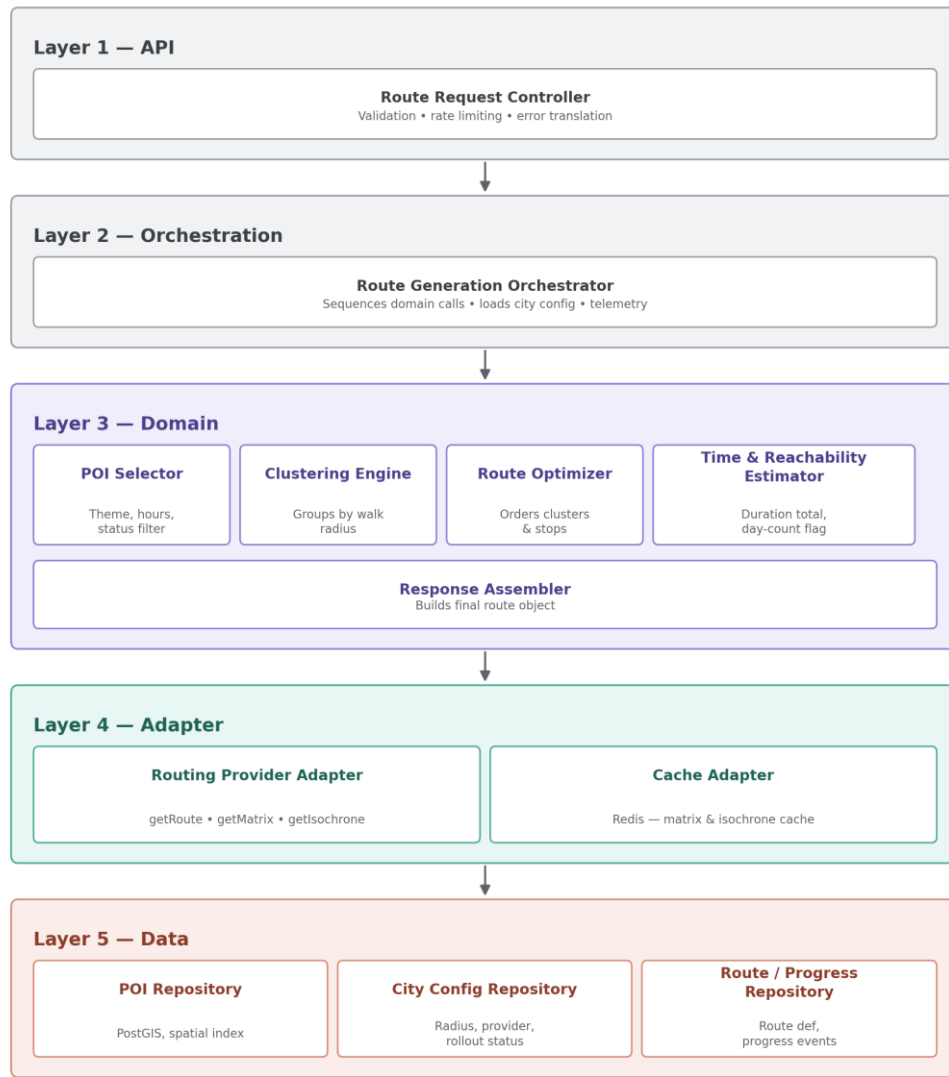


Figure 1 — Five-layer system architecture

3. Layer Responsibilities

Layer	Component	Responsibility
1 · API	Route Request Controller	Validates requests, rate-limits, translates errors. Not built this phase — see Section 9.
2 · Orchestration	Route Generation Orchestrator	Sequences Domain calls in order, loads city config, owns graceful degradation and telemetry.
3 · Domain	POI Selector	Queries eligible, published POIs by theme, opening hours, and city.
3 · Domain	Clustering Engine	Groups POIs into walkable clusters by configurable radius. Pure function.
3 · Domain	Route Optimizer	Orders clusters (driving) and stops within a cluster (walking).
3 · Domain	Time & Reachability Estimator	Sums travel + dwell time; sets the day-count flag via an isochrone check.

Layer	Component	Responsibility
3 · Domain	Response Assembler	Builds the final route object: segments, mode tags, checkpoint radii.
4 · Adapter	Routing Provider Adapter	Single interface to the routing engine. Normalizes responses. The provider-swap point.
4 · Adapter	Cache Adapter	Wraps Redis. Checked before every Adapter call. Owns the nightly matrix warm job.
5 · Data	POI Repository	PostGIS. Spatial, status, and category queries.
5 · Data	City Config Repository	Per-city settings. Cached in memory with a short refresh interval.
5 · Data	Route / Progress Repository	Immutable route definitions, kept separate from mutable progress state.

4. System Workflow & Data Flow

A route generation request flows through the stack in one direction, top to bottom. The efficiency rule that governs this layer: the routing provider's matrix endpoint is called once per request — covering all cluster and stop pairs in a single call — never once per pair.

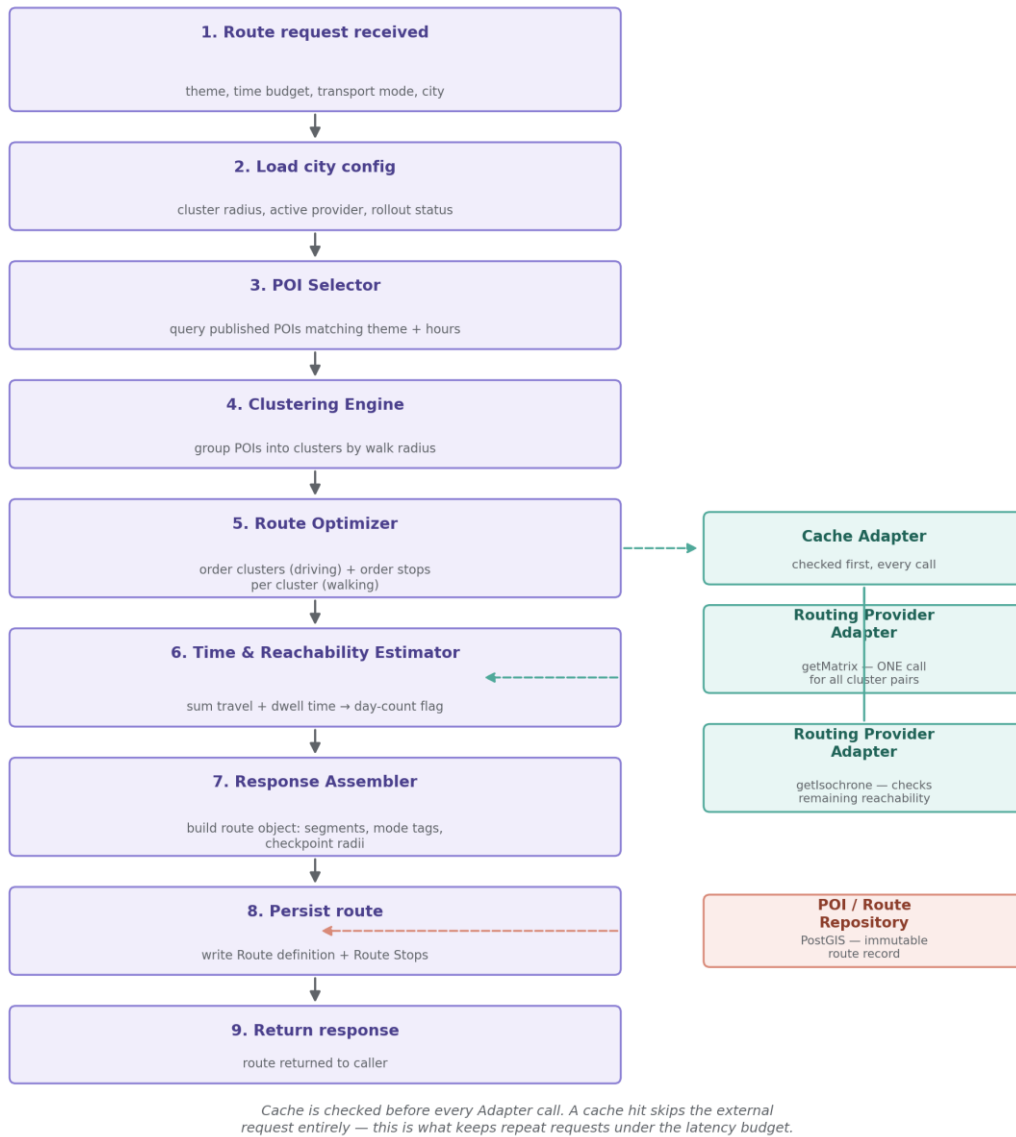


Figure 2 — Routing workflow and data flow

Latency budget

- Target: under 800ms end-to-end on a typical (cache-hit) request.
- Ceiling: 1.5s on a cold-cache request needing both a matrix and an isochrone call.
- Nearest-neighbor + 2-opt ordering itself costs under 1ms — the two external calls dominate the budget, which is why caching is the primary lever, not algorithm choice.

5. Core Algorithms & Routing Logic

Clustering

Fixed-radius greedy grouping: POIs within the city's configured radius (default 500m) of each other join a cluster. $O(n^2)$ at $n \approx 20$ –30 candidate POIs is effectively instant — no spatial clustering library is needed at this scale.

Route ordering

Nearest-neighbor construction followed by a single 2-opt improvement pass, run entirely in-memory against the cached distance/duration matrix. This gives a route within roughly 5% of optimal for the stop counts involved (single digits to low teens per cluster) in well under a millisecond. An exact TSP solver is deliberately not used — it costs exponentially more for a gain no user would perceive.

Hybrid transport logic

- Inter-cluster legs: driving profile, anchor point to anchor point.
- Intra-cluster legs: walking profile, ordering the stops within an arrived-at cluster.
- Every output segment is tagged with its mode (drive / walk) — this tagging is what lets the AR/UI layer render "drive here, then walk this loop" without knowing how the segment was generated.

Time & reachability estimation

- Total duration = Σ travel-time segments (from the cached matrix) + Σ per-POI dwell time (from POI data).
- Day-count flag: derived from an isochrone check — if remaining unvisited POIs fall outside the reachable area for the remaining time budget, the flag is set to 2+.

6. APIs & External Services

Service	Role	Notes
GraphHopper Directions API (hosted)	Primary routing provider — matrix, isochrone, directions	Free plan, no credit card required. Non-commercial use restriction on the free tier.
OpenRouteService (hosted)	Alternate / backup routing provider	Free tier: 2,500 requests/day. Open-source, same endpoint set, no card needed to sign up.
Self-hosted OSRM / GraphHopper / Valhalla	Future scale path once hosting budget exists	Runs on the Algeria OSM extract (~284MB). Near-zero marginal cost. Valhalla preferred if elevation-aware pedestrian costing is needed later.
Overpass API (OpenStreetMap)	Primary POI bootstrap source	Free, no card. One-time pull per city; results land as draft / api_seeded, reviewed before publishing.
Google Places API	Supplemental POI source only	Requires a payment card even for free-tier use — use only for specific gaps Overpass misses.
Traffic API	Not in scope for v1	Static walking/driving estimates are sufficient for a pedestrian/stamp-rally use case.

7. Data Models & Key Interfaces

Schema (PostgreSQL / PostGIS)

```
-- REGIONS — wilayas, administrative scaffold for phased rollout
CREATE TABLE regions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  name_ar TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- CITIES — also holds per-city routing / rollout config
CREATE TYPE rollout_status AS ENUM ('planning', 'pilot', 'active');
CREATE TYPE routing_provider AS ENUM (
  'graphhopper', 'ors',
  'self_hosted_osrm', 'self_hosted_graphhopper', 'self_hosted_valhalla'
);

CREATE TABLE cities (
```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
region_id UUID REFERENCES regions(id),
name TEXT NOT NULL,
name_ar TEXT,
name_fr TEXT,
bounding_box GEOGRAPHY(POLYGON, 4326),
cluster_radius_meters INT NOT NULL DEFAULT 500,
active_routing_provider routing_provider NOT NULL DEFAULT 'graphhopper',
rollout_status rollout_status NOT NULL DEFAULT 'planning',
feature_flags JSONB NOT NULL DEFAULT '{}',
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
deleted_at TIMESTAMPTZ
);

-- CATEGORIES — lookup table, carries display metadata
CREATE TABLE categories (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  key TEXT NOT NULL UNIQUE,
  label_fr TEXT NOT NULL,
  label_ar TEXT NOT NULL,
  label_en TEXT NOT NULL,
  icon_ref TEXT,
  color_hex TEXT
);

-- POIS
CREATE TYPE poi_source AS ENUM ('team_seeded', 'api_seeded', 'user_submitted', 'ministry_provided');
CREATE TYPE poi_status AS ENUM ('draft', 'verified', 'published');

CREATE TABLE pois (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  city_id UUID NOT NULL REFERENCES cities(id),
  category_id UUID NOT NULL REFERENCES categories(id),
  name_fr TEXT, name_ar TEXT, name_en TEXT,
  description_fr TEXT, description_ar TEXT, description_en TEXT,
  location GEOGRAPHY(POINT, 4326) NOT NULL,
  opening_hours_raw TEXT, -- OSM opening_hours syntax, parseable
  avg_visit_duration_minutes INT NOT NULL DEFAULT 15,
  ar_content_id UUID, -- FK added once AR module owns its table
  stamp_id UUID, -- FK added once gamification module owns its table
  external_ref TEXT, -- e.g. 'osm:node/123456' — dedup / re-sync anchor
  source poi_source NOT NULL DEFAULT 'team_seeded',
  status poi_status NOT NULL DEFAULT 'draft',
  verified_by TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ
);
CREATE INDEX idx_pois_location ON pois USING GIST (location);
CREATE INDEX idx_pois_city_status_category ON pois (city_id, status, category_id) WHERE deleted_at IS NULL;
CREATE INDEX idx_pois_external_ref ON pois (external_ref) WHERE external_ref IS NOT NULL;

-- ROUTES — immutable once generated
CREATE TYPE transport_mode AS ENUM ('walking', 'driving', 'hybrid');

CREATE TABLE routes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  city_id UUID NOT NULL REFERENCES cities(id),
  session_id UUID, -- nullable: supports anonymous demo usage
  user_id UUID, -- nullable until auth exists
  theme TEXT NOT NULL,
  time_budget_minutes INT NOT NULL,
  transport_mode transport_mode NOT NULL DEFAULT 'hybrid',
  segments JSONB NOT NULL, -- opaque per-leg geometry / mode / duration
  estimated_total_duration_minutes INT NOT NULL,
  day_count_flag INT NOT NULL DEFAULT 1,
  generated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ROUTE_STOPS — queryable, kept out of the segments JSON on purpose
CREATE TABLE route_stops (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  route_id UUID NOT NULL REFERENCES routes(id) ON DELETE CASCADE,
  poi_id UUID NOT NULL REFERENCES pois(id),
  sequence_order INT NOT NULL,
  cluster_id INT NOT NULL
);

```



```

CREATE INDEX idx_route_stops_route ON route_stops (route_id);
CREATE INDEX idx_route_stops_poi ON route_stops (poi_id);    -- powers data-flywheel queries

-- PROGRESS – mutable, kept separate from the immutable route definition
CREATE TYPE progress_status AS ENUM ('in_progress', 'completed', 'abandoned');

CREATE TABLE progress (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  route_id UUID NOT NULL REFERENCES routes(id),
  status progress_status NOT NULL DEFAULT 'in_progress',
  started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  last_updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  completed_at TIMESTAMPTZ
);

-- PROGRESS_EVENTS – append-only checkpoint log
CREATE TABLE progress_events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  progress_id UUID NOT NULL REFERENCES progress(id) ON DELETE CASCADE,
  poi_id UUID NOT NULL REFERENCES pois(id),
  arrived_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_progress_events_progress ON progress_events (progress_id);

```

Adapter and Domain interface contracts

```

// Adapter layer – the single seam to the routing engine
interface RoutingProviderAdapter {
  getRoute(stops: Coordinate[], mode: 'walking' | 'driving'): RouteResult
  getMatrix(points: Coordinate[], mode: 'walking' | 'driving'): DurationMatrix
  getIsochrone(origin: Coordinate, timeBudgetMinutes: number,
    mode: 'walking' | 'driving'): IsochronePolygon
}

interface CacheAdapter {
  getMatrix(cityId: string, mode: string): DurationMatrix | null
  setMatrix(cityId: string, mode: string, matrix: DurationMatrix): void
  getIsochrone(poiId: string, timeBucket: number, mode: string): IsochronePolygon | null
  setIsochrone(poiId: string, timeBucket: number, mode: string, polygon: IsochronePolygon): void
}

// Domain layer – pure functions, no direct provider or DB access
ClusteringEngine.cluster(pois: POI[], radiusMeters: number): Cluster[]

RouteOptimizer.orderClusters(clusters: Cluster[], matrix: DurationMatrix): Cluster[]
RouteOptimizer.orderStopsWithinCluster(cluster: Cluster, matrix: DurationMatrix): POI[]

TimeEstimator.estimate(
  orderedStops: POI[],
  matrix: DurationMatrix,
  dwellTimes: Record<string, number>
): { totalMinutes: number; dayCountFlag: number }

ResponseAssembler.assemble(
  orderedClusters: Cluster[],
  segments: Segment[],
  estimate: { totalMinutes: number; dayCountFlag: number }
): RouteResponse

```

8. Feature Specifications

Feature	Description
Multi-stop hybrid route generation	Ordered route across selected/eligible POIs, combining driving (inter-cluster) and walking (intra-cluster) segments.
Theme & time-budget filtering	POI Selector filters by theme, category, and city; Time Estimator checks fit against the user's stated time budget.
Day-count / extended-trip flag	Isochrone-based check on remaining POIs once the time budget is exhausted; flags when a rally needs 2+ days.

Feature	Description
Checkpoint output for AR handoff	Response includes ordered waypoints and a per-POI checkpoint radius for the separate AR trigger service to consume.
Matrix & isochrone caching	Cache-first on every Adapter call; a nightly scheduled job refreshes each active city's full POI matrix ahead of demand.
Quota-aware provider handling	Adapter enforces a timeout and falls back to the last cached matrix on provider failure or slowness.

9. Implementation Roadmap & Development Order

Bottom-up by dependency, not strictly by layer number — Data and Adapter are independent leaf branches and can be built in either order or in parallel; Adapter is sequenced first below because it carries the highest external uncertainty.

1. Define contracts. POI record shape, City Config fields, Routing Provider Adapter method signatures (Section 7). Design only, no code.

2. Adapter layer. Implement against a real provider using 2–3 hardcoded coordinate pairs in the pilot city. Validate `getRoute` and `getMatrix`. Cache Adapter can start as an in-memory dictionary; Redis is a later swap behind the same interface.

3. Data layer. Schema plus repository methods, seeded with 5–10 hand-authored fixture POIs. Test repository query methods directly against fixture data.

4. Domain layer. In dependency order — Clustering Engine and Response Assembler first (pure, no dependencies), then POI Selector (needs Data), then Route Optimizer (needs the Adapter's matrix call), then Time & Reachability Estimator last (needs both Adapter isochrone and Data dwell times). Unit test each before moving to the next.

5. Orchestration. Wire the Domain components together in sequence; pull City Config from Data. This is the first point the whole pipeline runs as one call.

6. Minimal driver script. Calls the Orchestrator with a realistic sample request and prints the result — validates the full pipeline end-to-end. Deliberately substitutes for Layer 1 during this phase; becomes the body of the first controller endpoint almost verbatim once a real API is built.

10. Edge Cases & Assumptions

Data

- OSM tourist-site tagging is sparser than road coverage outside major cities — manual curation is required before a POI reaches published status.
- POIs are currently sourced via team seeding plus a one-time Overpass pull; results are marked `draft / api_seeded` until manually verified.
- Multilingual fields (French / Arabic / English) are required from the start; opening hours are stored in parseable OSM syntax, not free text.

Routing & constraints

- Parking availability and one-way/restricted-zone streets are explicitly out of scope for the demo phase. They are isolated to future logic inside the driving-segment handling in the Adapter and Response Assembler, so handling them later does not require a redesign.
- No international payment card is currently available — GraphHopper's no-card free plan and OpenRouteService's no-card signup are the only routing providers in scope until this changes.
- Provider free-tier quotas are limited (per-minute credits, or 2,500 requests/day). Cache-first design and the nightly warm job are mandatory to stay within quota during development and the demo, not optional optimizations.

Operational

- GPS drift in dense old-city areas can cause false or missed checkpoint detection — checkpoint radius is per-POI configurable, not global; dwell-time-before-confirm logic belongs to the separate AR trigger service.
- Offline / low-connectivity handling is a known future requirement, not solved in this phase.
- Cluster radius (default 500m) is a per-city config value expected to need tuning once real pilot-city data is available.

11. Basic Testing Strategy

Level	What & how
Unit — pure functions	Clustering Engine, Route Optimizer's ordering step (against a fixed known matrix fixture), Response Assembler, Time & Reachability Estimator arithmetic. Fast, fixture-based, run on every change.
Repository	POI Repository and City Config Repository tested against the 5–10 hand-authored fixture POIs.
Adapter	Routing Provider Adapter validated against the real provider using 2–3 fixed coordinate pairs (manual/one-time integration check). A mock/stub adapter is used for all automated test runs to avoid consuming free-tier quota.
Cache	Assert a cache hit skips the underlying Adapter call entirely (mock call-count assertion).
End-to-end	The Section 9 driver script, run against the fixture city and fixture POIs, checked against an expected route shape: correct segment count, mode tags present, plausible day-count flag.